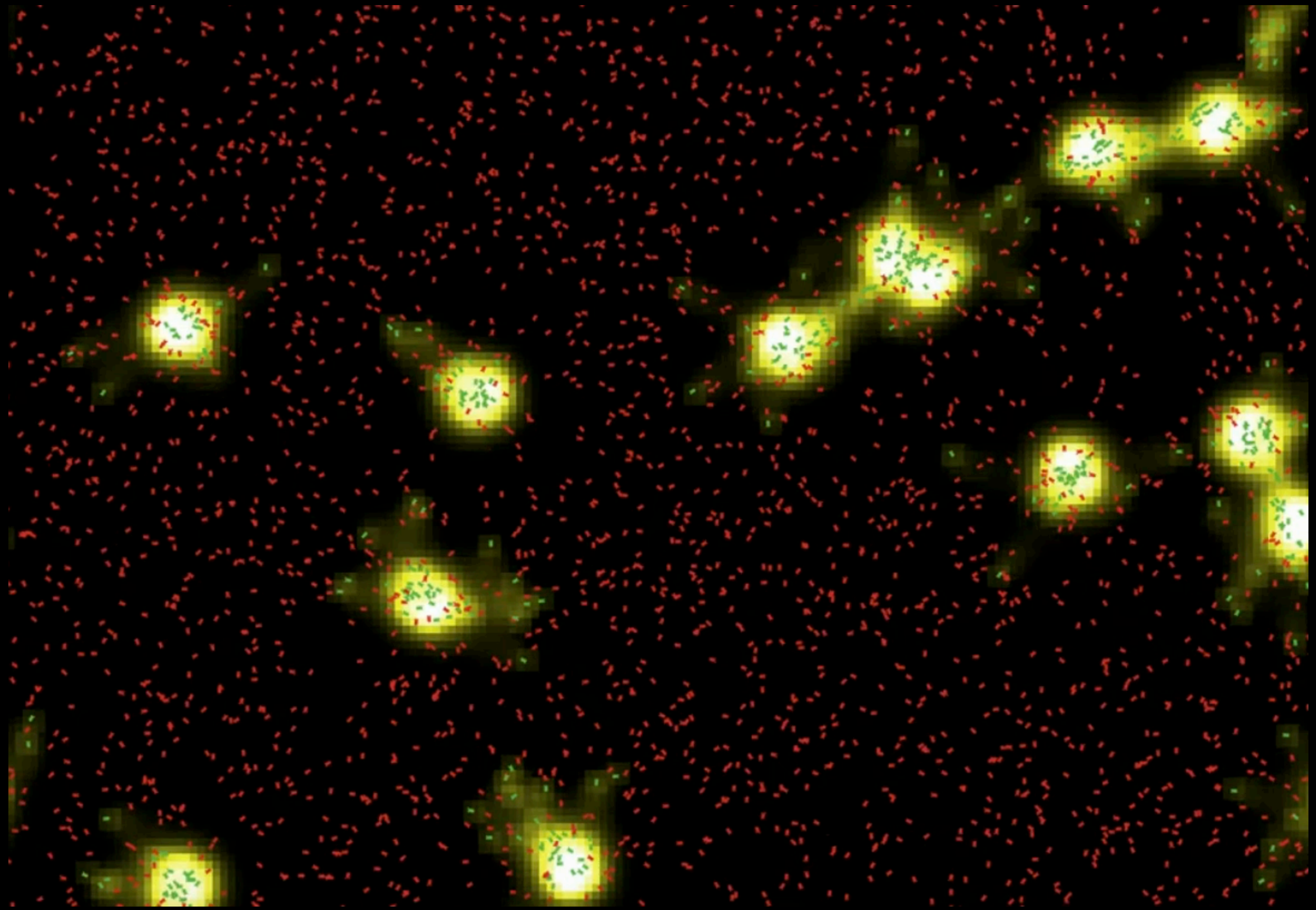
Bayesian approaches

Raoul Grouls, 26 Mei 2025

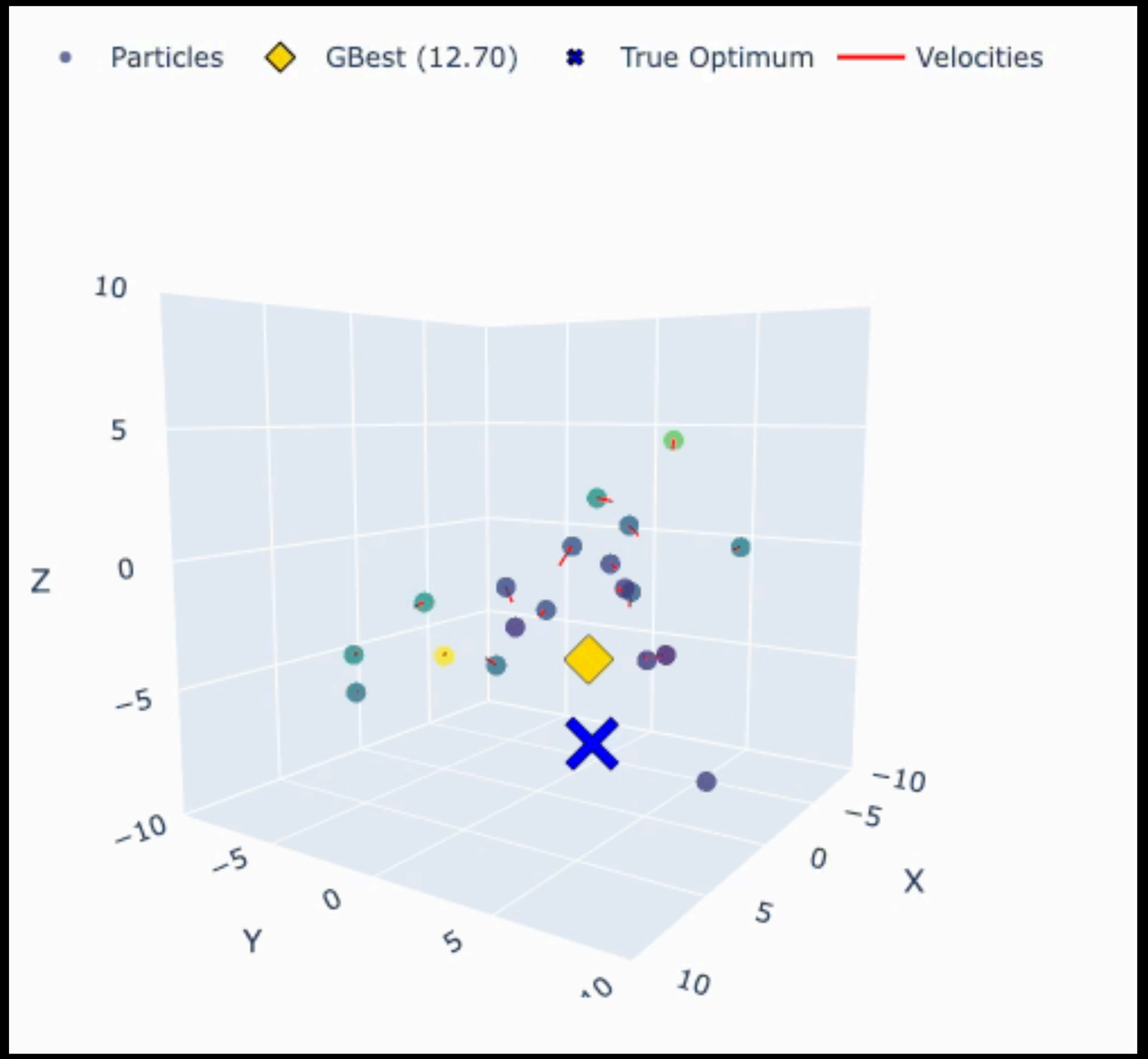
Two ways to solve problems

- Analytical
- With the help of random processes


מכון ויצמן למדע
A



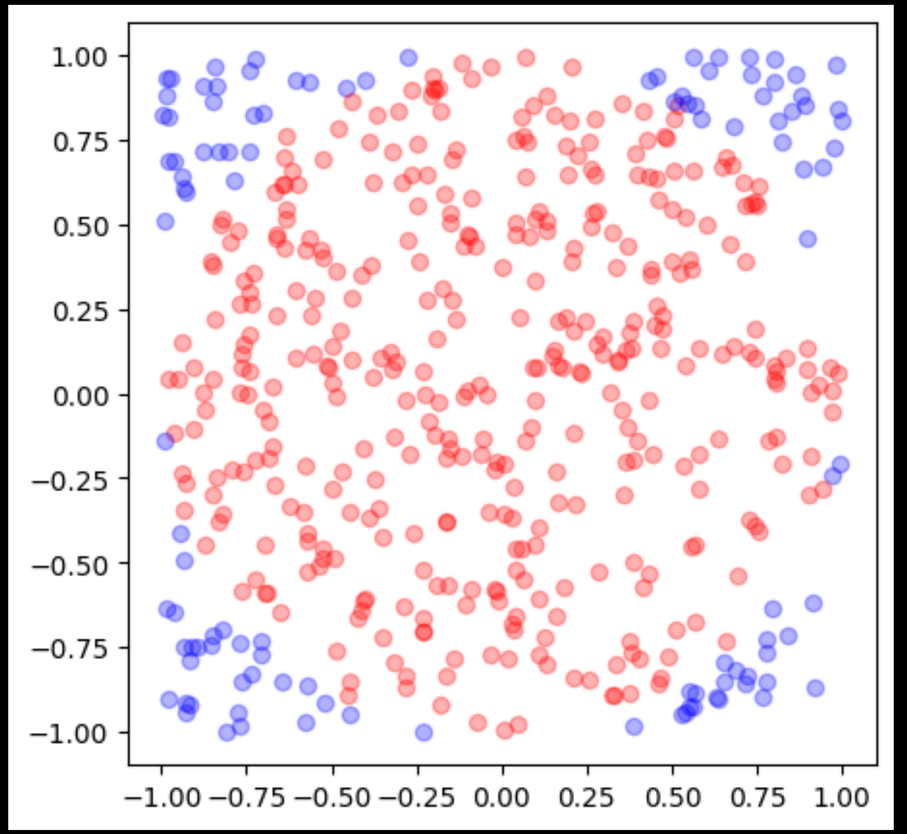
$$y = (x - 4)^2 + (y - 5)^2 + (z + 6)^2$$


Particles
GBest (12.70)
True Optimum
Velocities
10
5
0
−5
−10
Z
−10
−5
0
5
10
Y
−10
−5
0
5
10
X

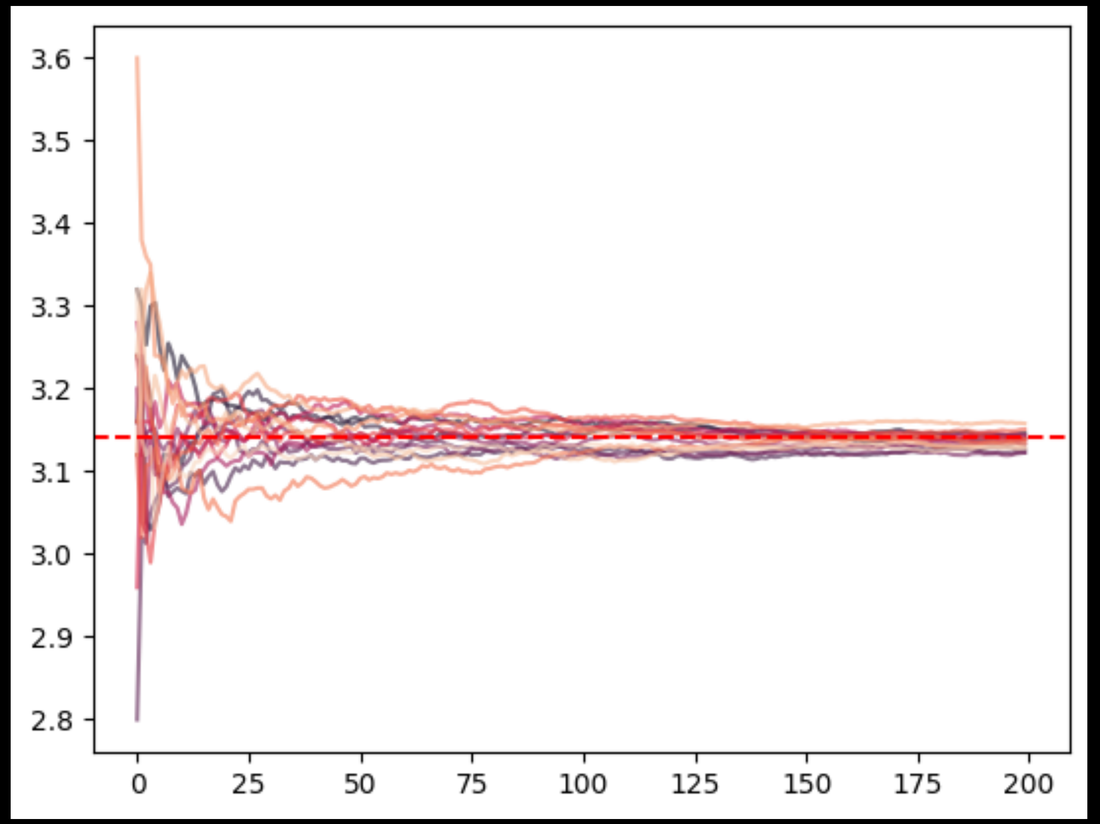
Exploit vs Explore

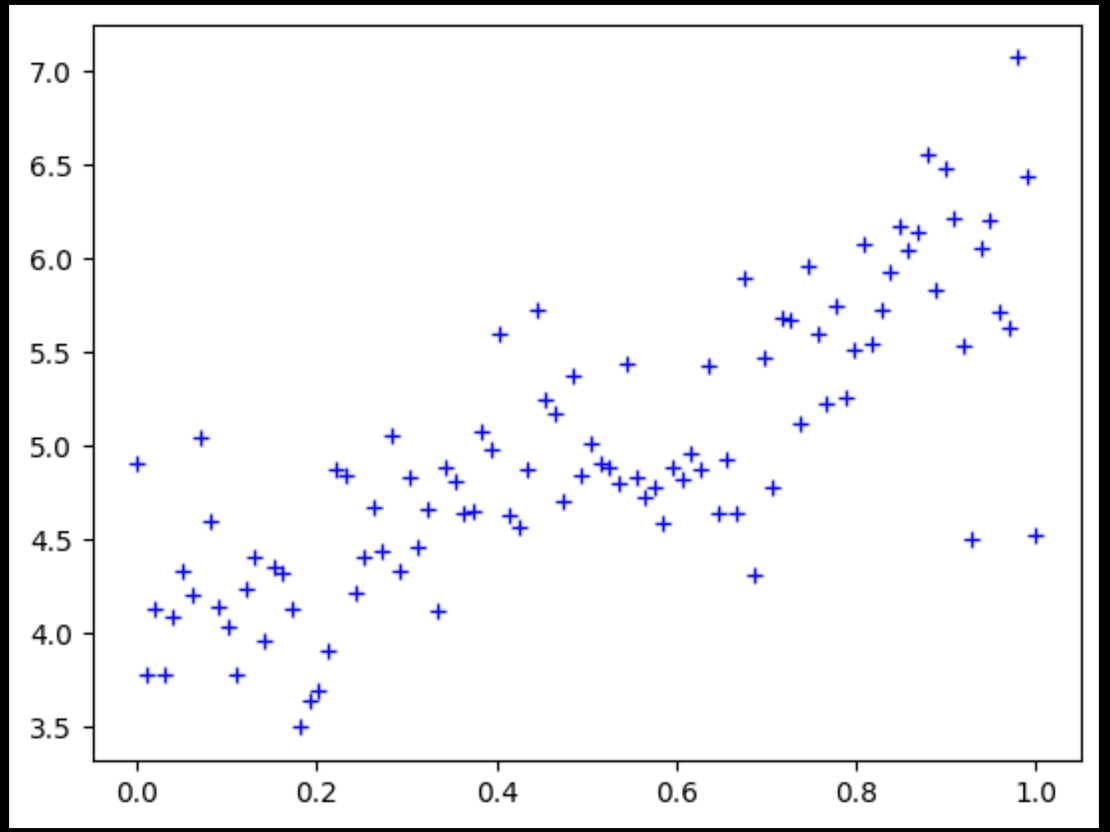
- Exploit: follow best practices
- Explore: do something random to explore new options

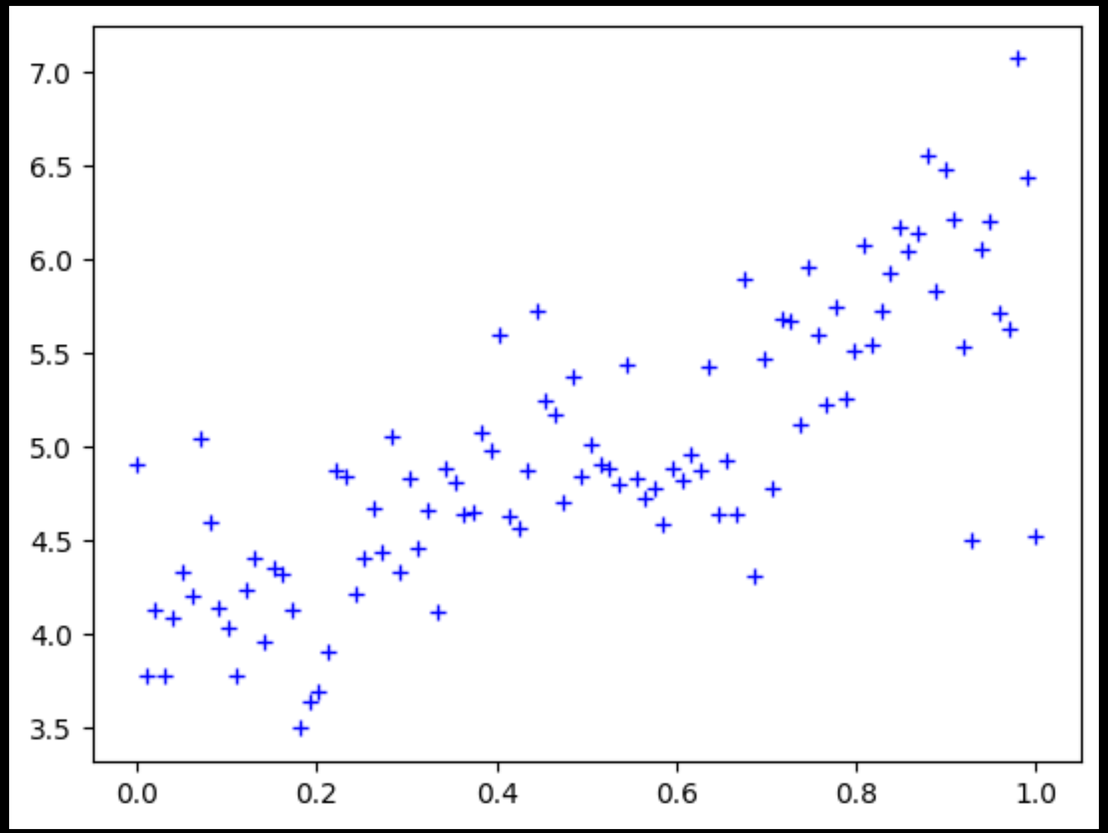
How to find π



Multiple runs increase accuracy




7.0
6.5
6.0
5.5
5.0
4.5
4.0
3.5
0.0
0.2
0.4
0.6
0.8
1.0



$$model(x) = ax + b$$

$$model(x) = ax + b$$

$$model(x) = ax + b$$



Variables

$$model(x) = ax + b$$

$$a \in \mathbb{R}$$

$$b \in \mathbb{R}$$

$$model(x) = ax + b$$

$$a \sim \mathcal{N}(\mu, \sigma)$$

$$model(x) = ax + b$$

$$a \sim \mathcal{N}(\mu, \sigma)$$

$$b \sim \mathcal{N}(\mu, \sigma)$$

$$model(x) = ax + b + noise$$

$$model(x) = ax + b + \mathcal{N}(\mu, \sigma)$$

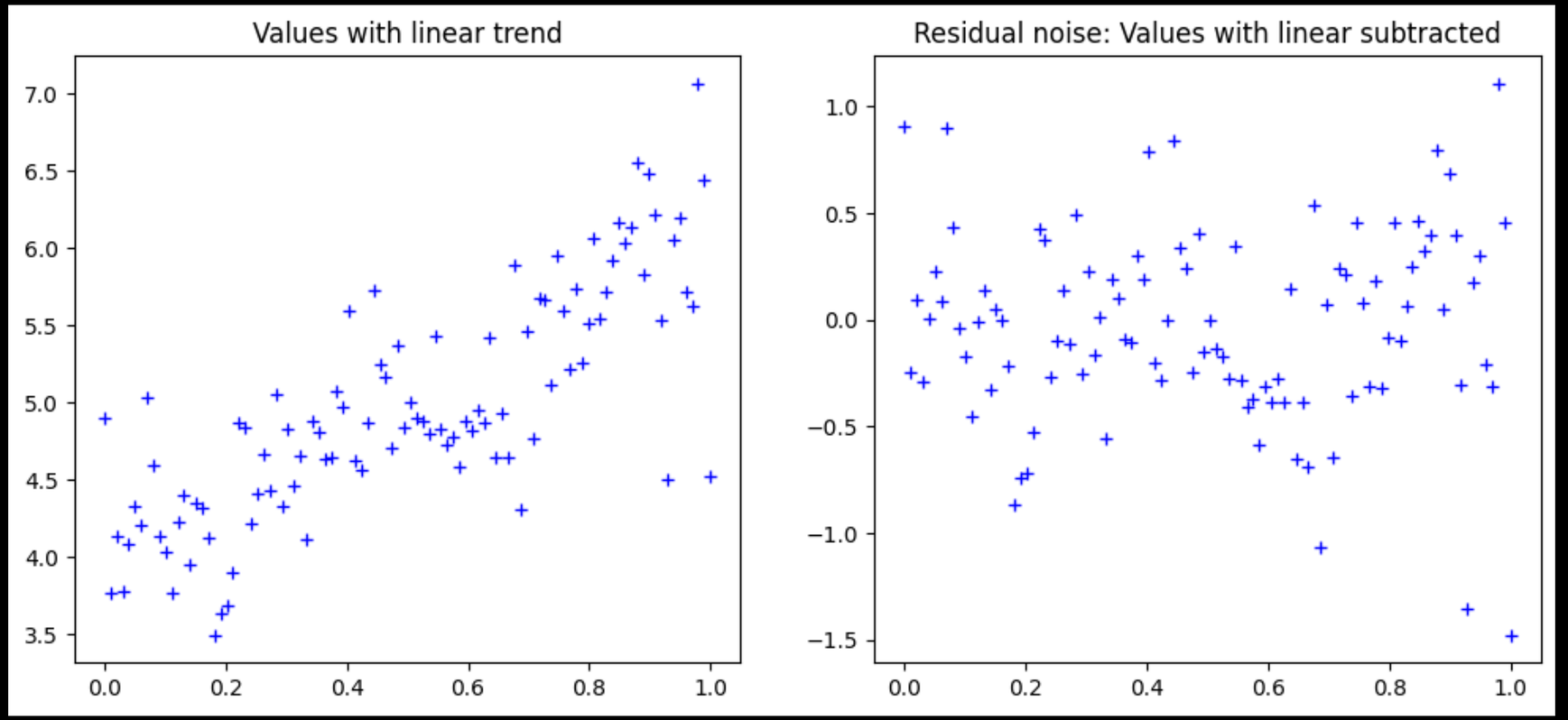
$$model(x) - (ax + b) = \mathcal{N}(\mu, \sigma)$$

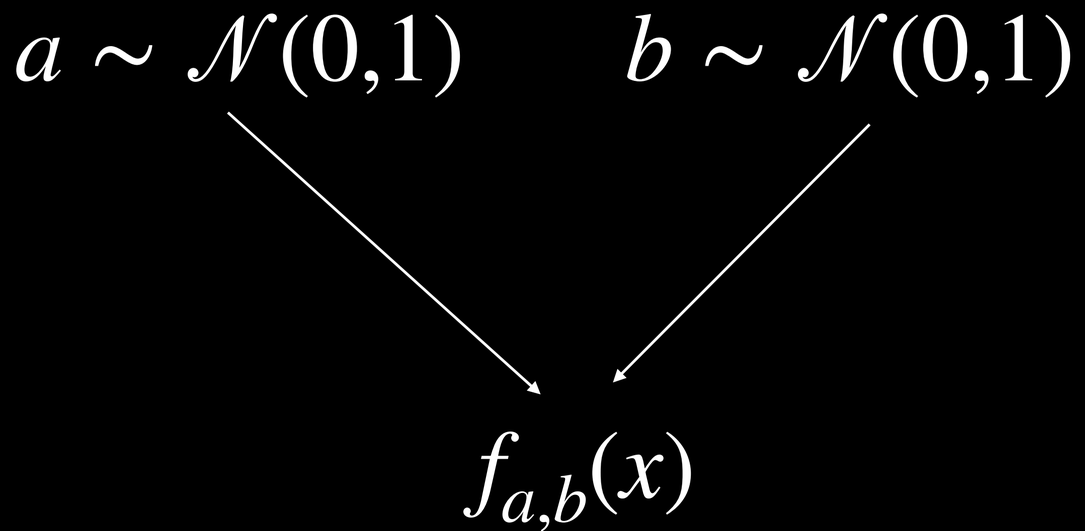
$$model(x) - (ax + b) = \mathcal{N}(\mu, \sigma)$$

$$model(x) = \mathcal{N}(ax + b, \sigma)$$

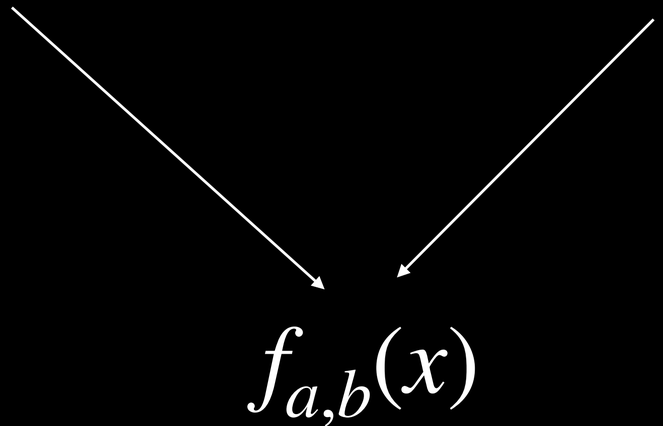
$$model(x) = \mathcal{N}\left(f(x), \sigma\right)$$

$$model(x) = \mathcal{N}\left(f_\theta(x), \sigma\right)$$

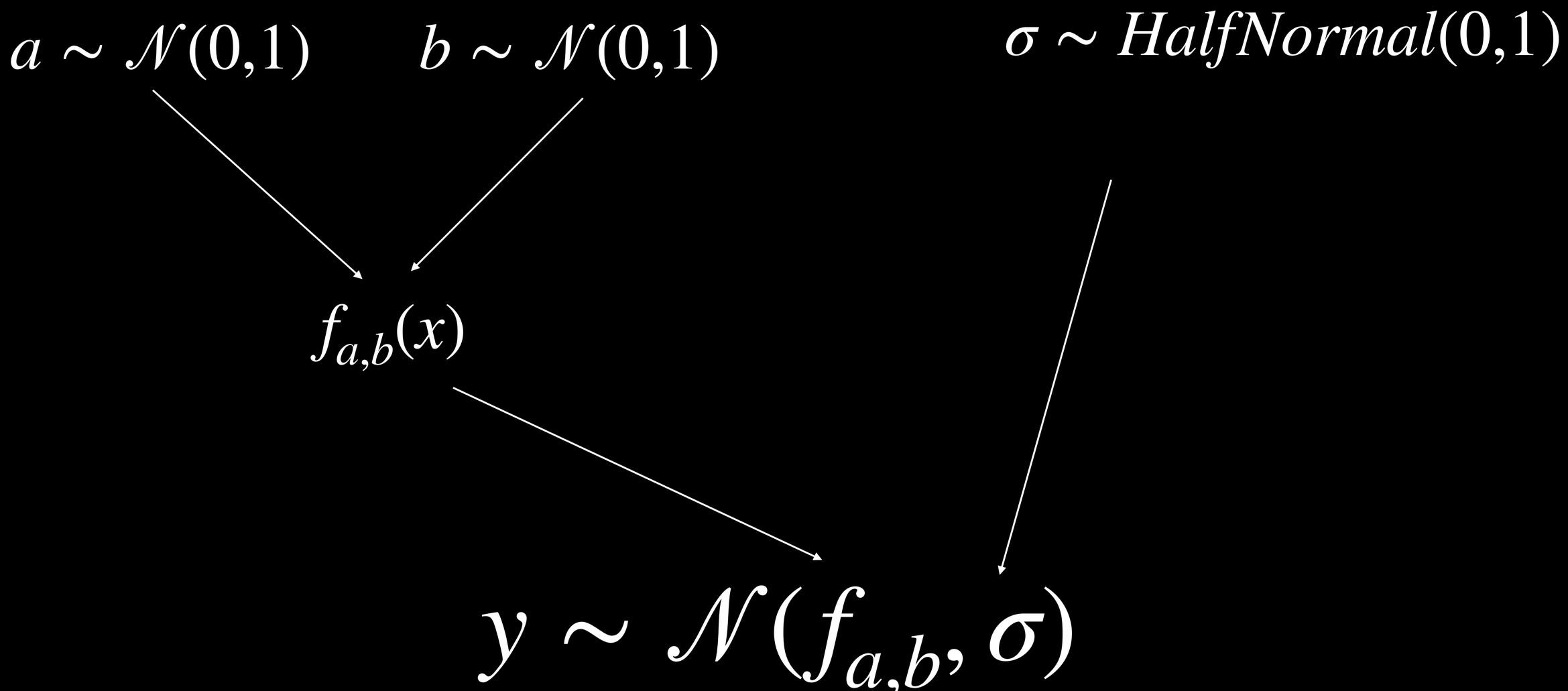

Values with linear trend
7.0
6.5
6.0
5.5
5.0
4.5
4.0
3.5
0.0
0.2
0.4
0.6
0.8
1.0
Residual noise: Values with linear subtracted
1.0
0.5
0.0
−0.5
−1.0
−1.5
0.0
0.2
0.4
0.6
0.8
1.0


$a \sim \mathcal{N}(0,1)$
$b \sim \mathcal{N}(0,1)$
$f_{a,b}(x)$

$a \sim \mathcal{N}(0,1)$ $b \sim \mathcal{N}(0,1)$ $\sigma \sim HalfNormal(0,1)$



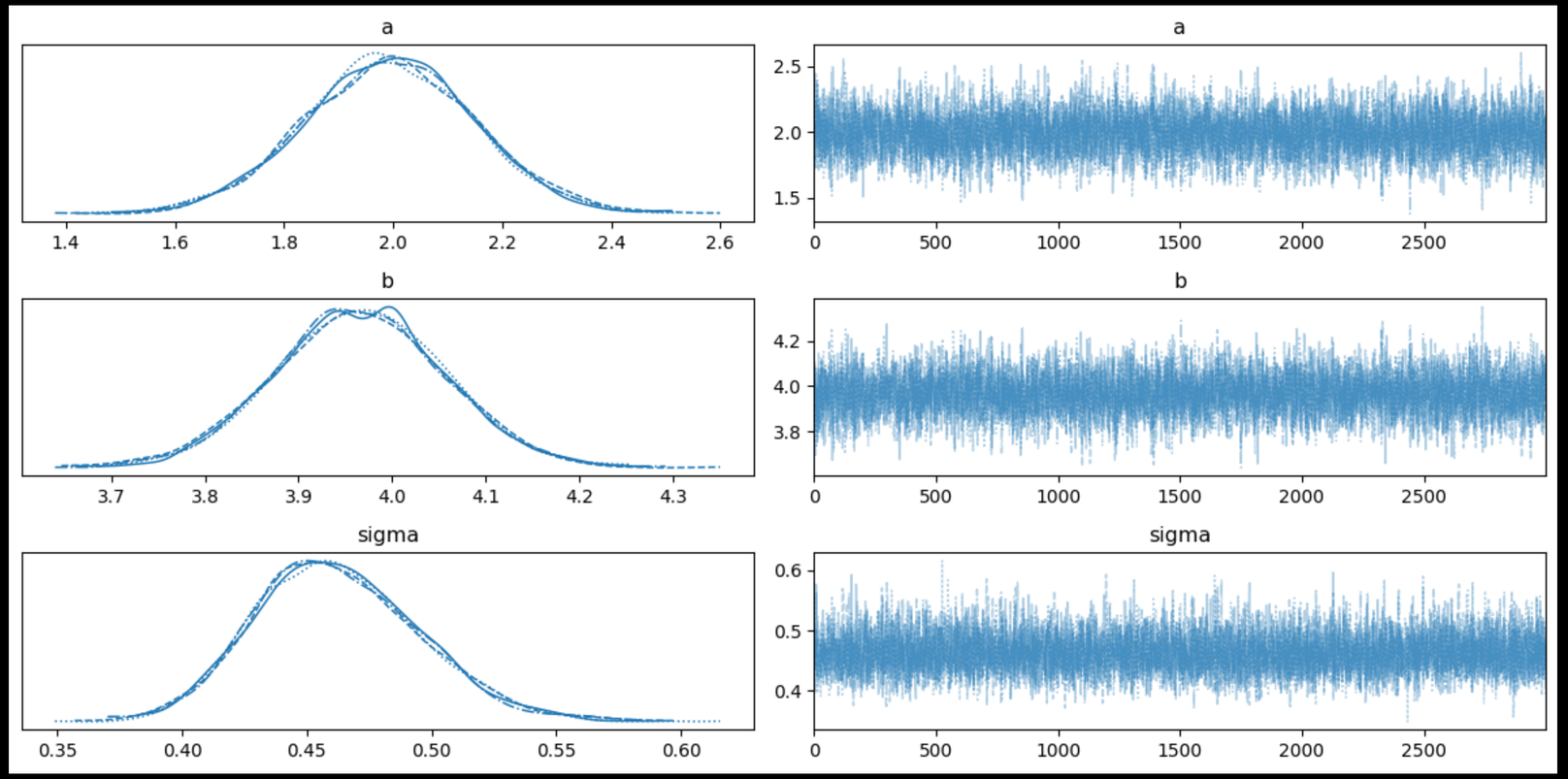
$f_{a,b}(x)$

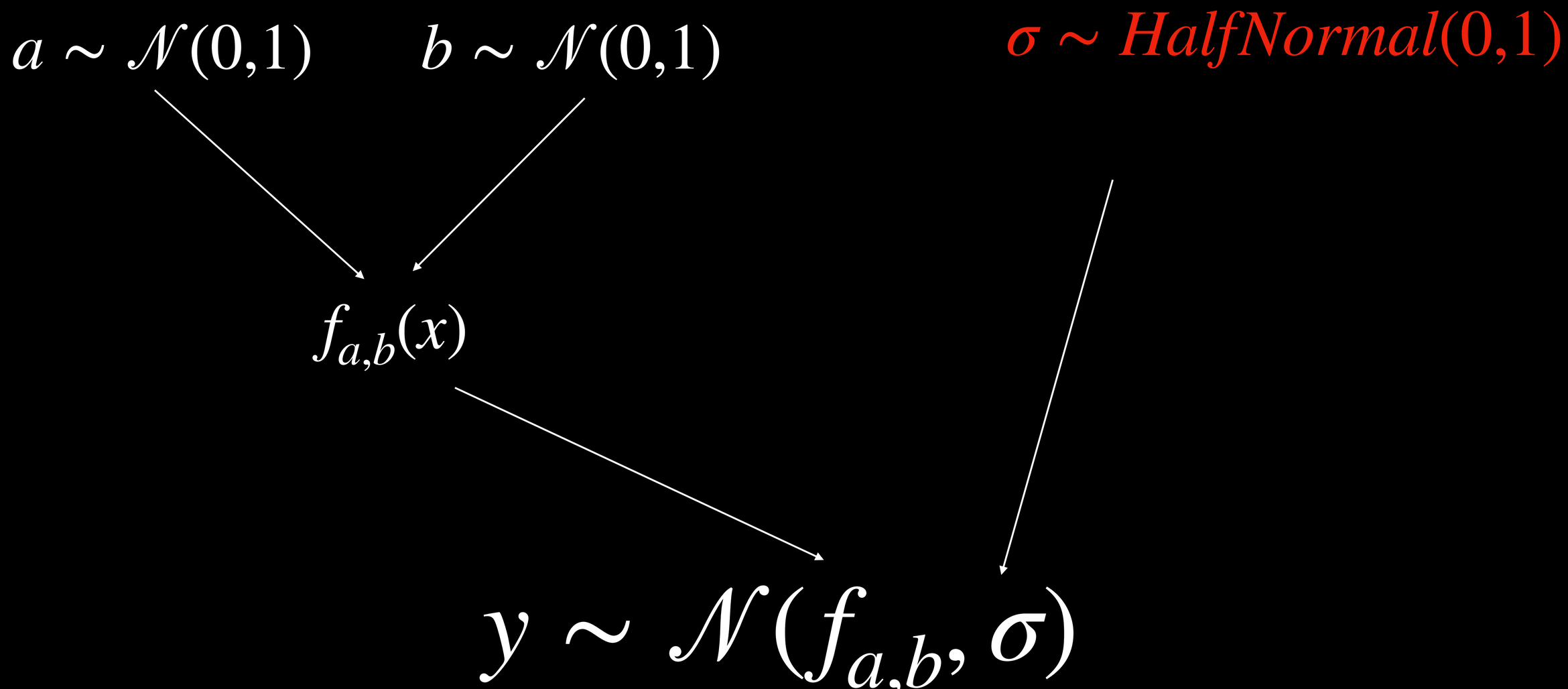

$a \sim \mathcal{N}(0,1)$
$b \sim \mathcal{N}(0,1)$
$\sigma \sim HalfNormal(0,1)$
$f_{a,b}(x)$
$y \sim \mathcal{N}(f_{a,b}, \sigma)$

Pymc implementation

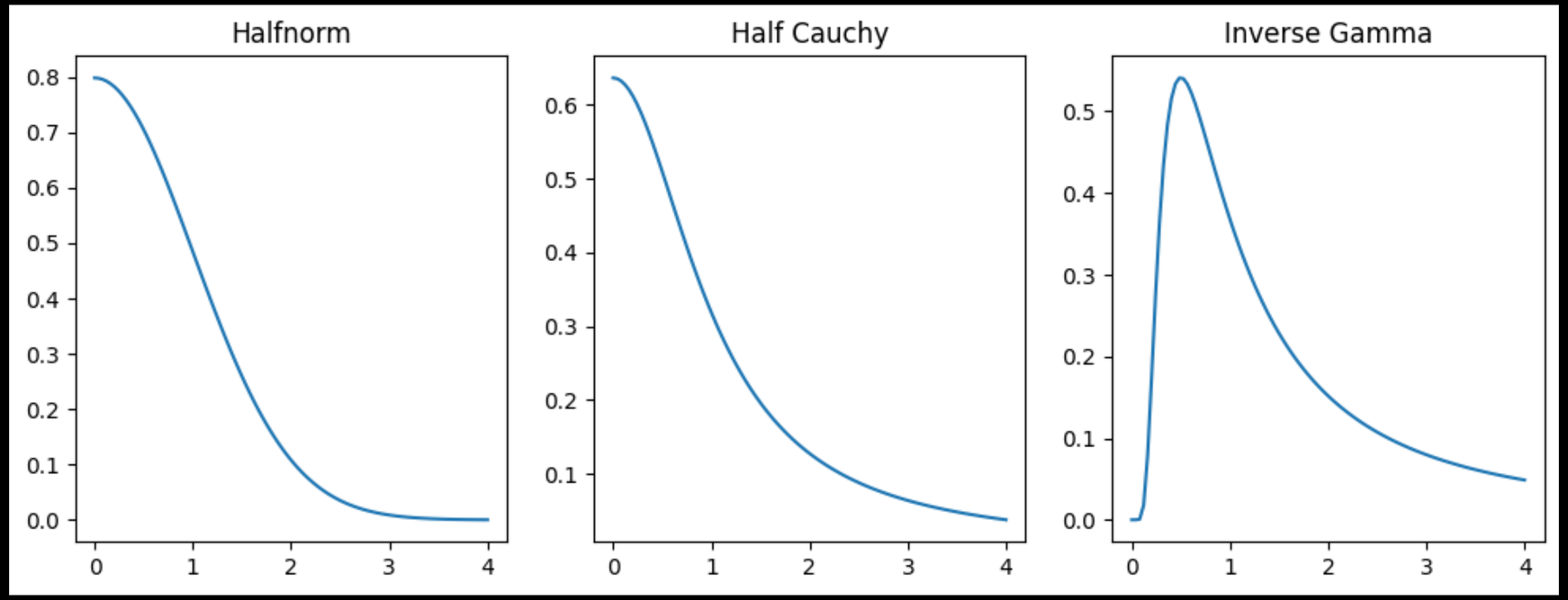
```
with model:
    sigma = pm.HalfNormal("sigma")
    predict = a * x + b
    estimate = pm.Normal("y", mu=predict, sigma=sigma, observed=y)
```

Pymc result

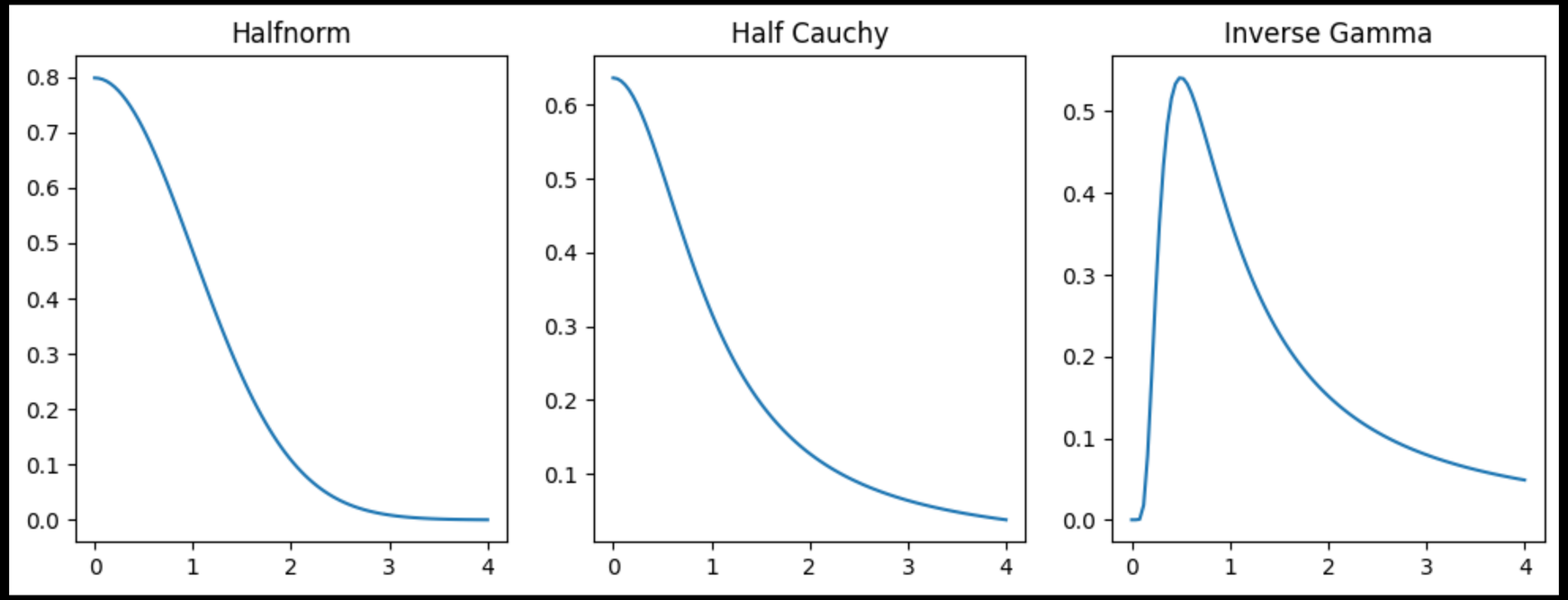

a
1.4
1.6
1.8
2.0
2.2
2.4
2.6
a
2.5
2.0
1.5
0
500
1000
1500
2000
2500
b
3.7
3.8
3.9
4.0
4.1
4.2
4.3
b
4.2
4.0
3.8
0
500
1000
1500
2000
2500
sigma
0.35
0.40
0.45
0.50
0.55
0.60
sigma
0.6
0.5
0.4
0
500
1000
1500
2000
2500


$a \sim \mathcal{N}(0,1)$
$b \sim \mathcal{N}(0,1)$
$\sigma \sim HalfNormal(0,1)$
$f_{a,b}(x)$
$y \sim \mathcal{N}(f_{a,b}, \sigma)$

Weakly informed priors

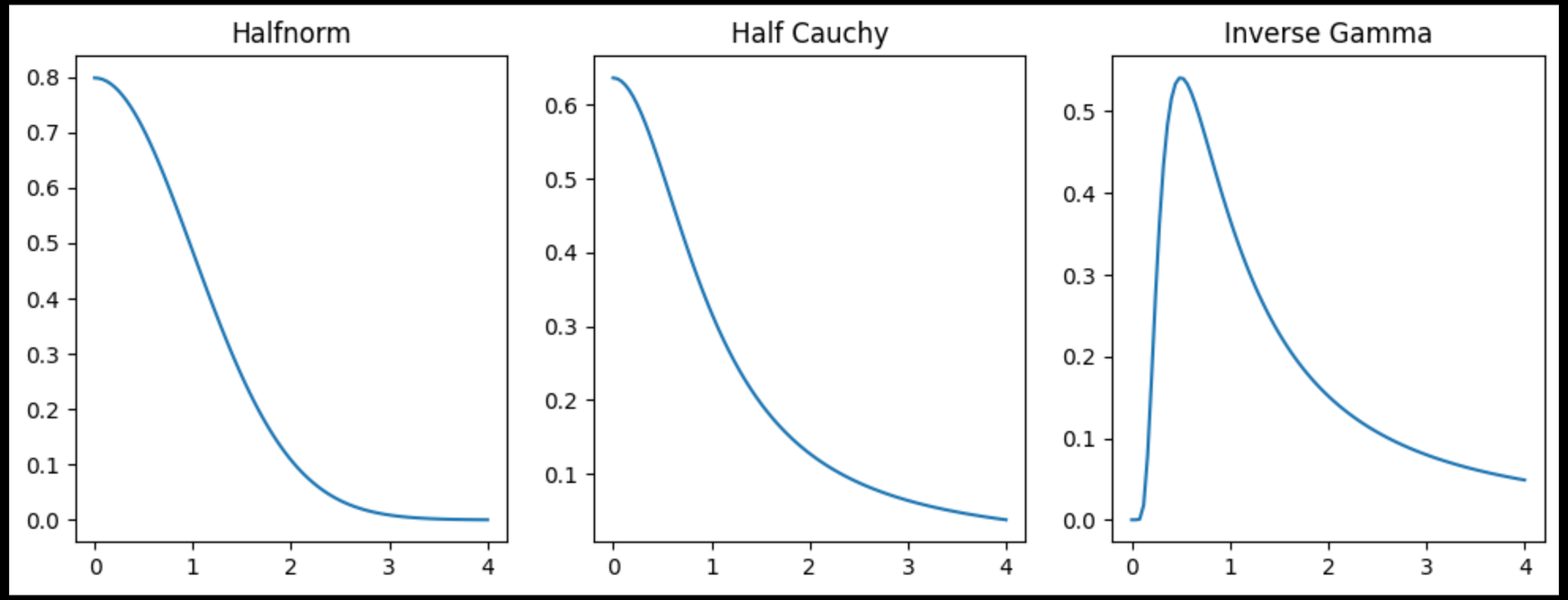


Weakly informed priors



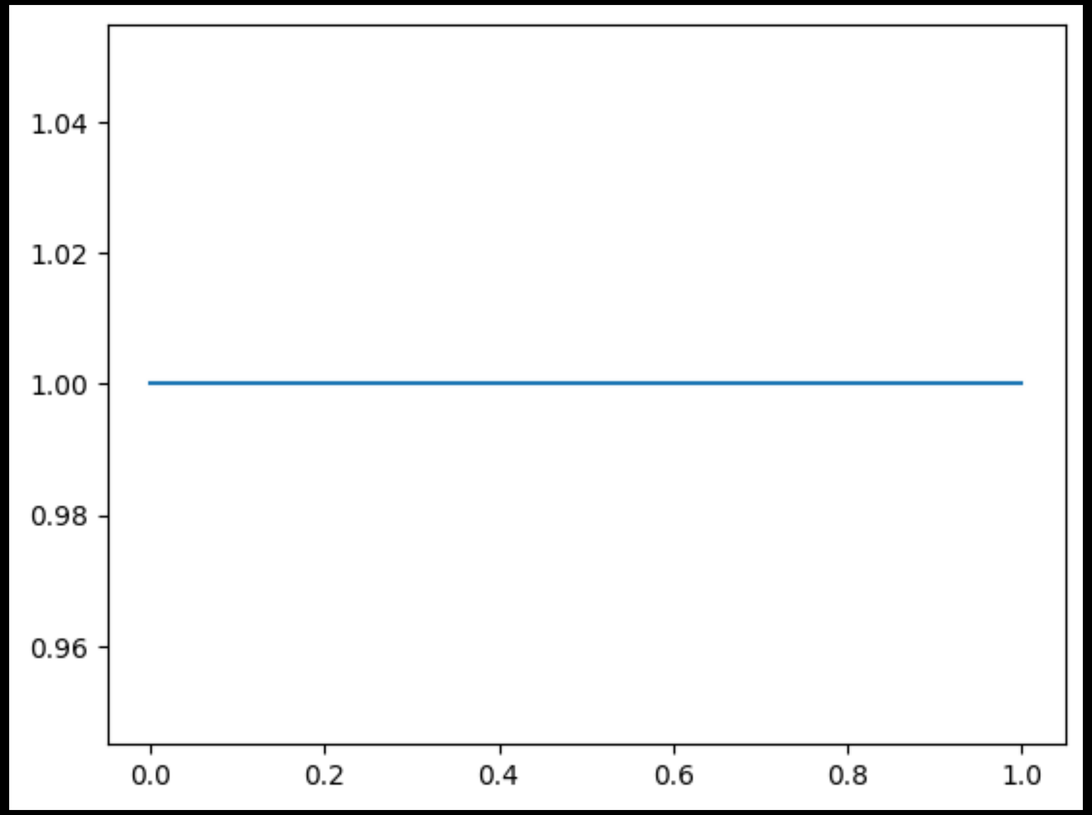
Longer tail

Weakly informed priors



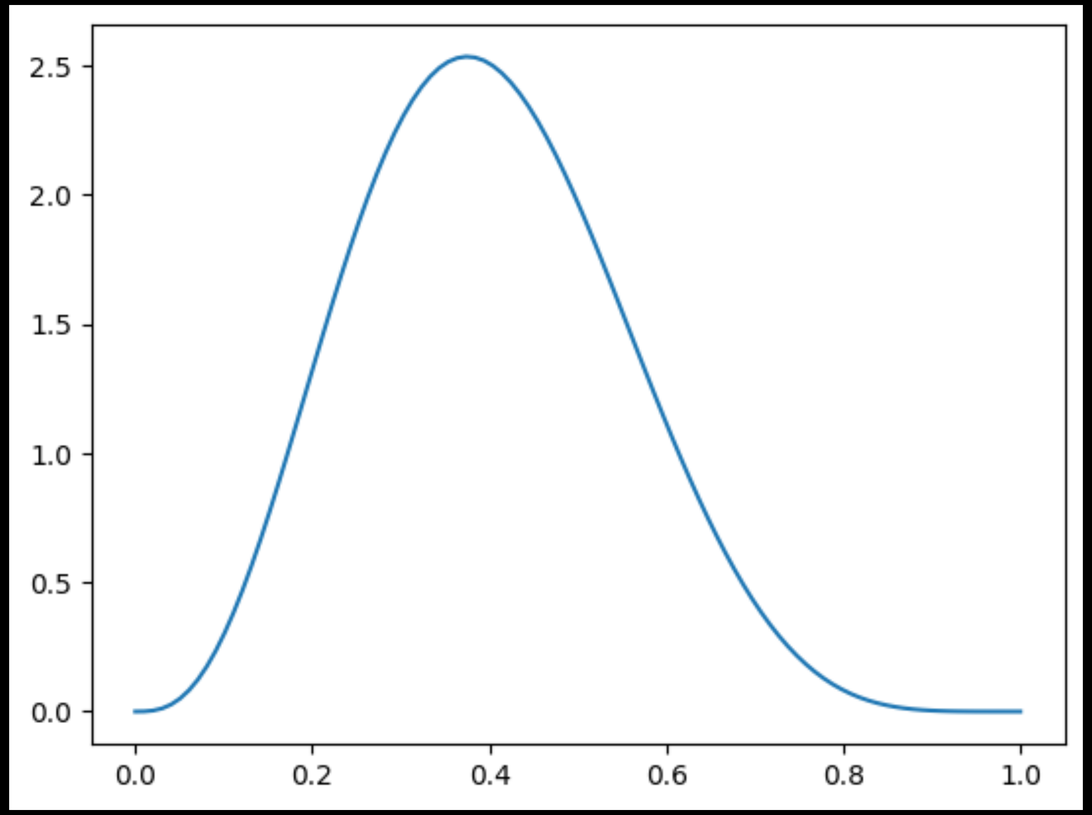
Longer tail, less zero

Beta distribution



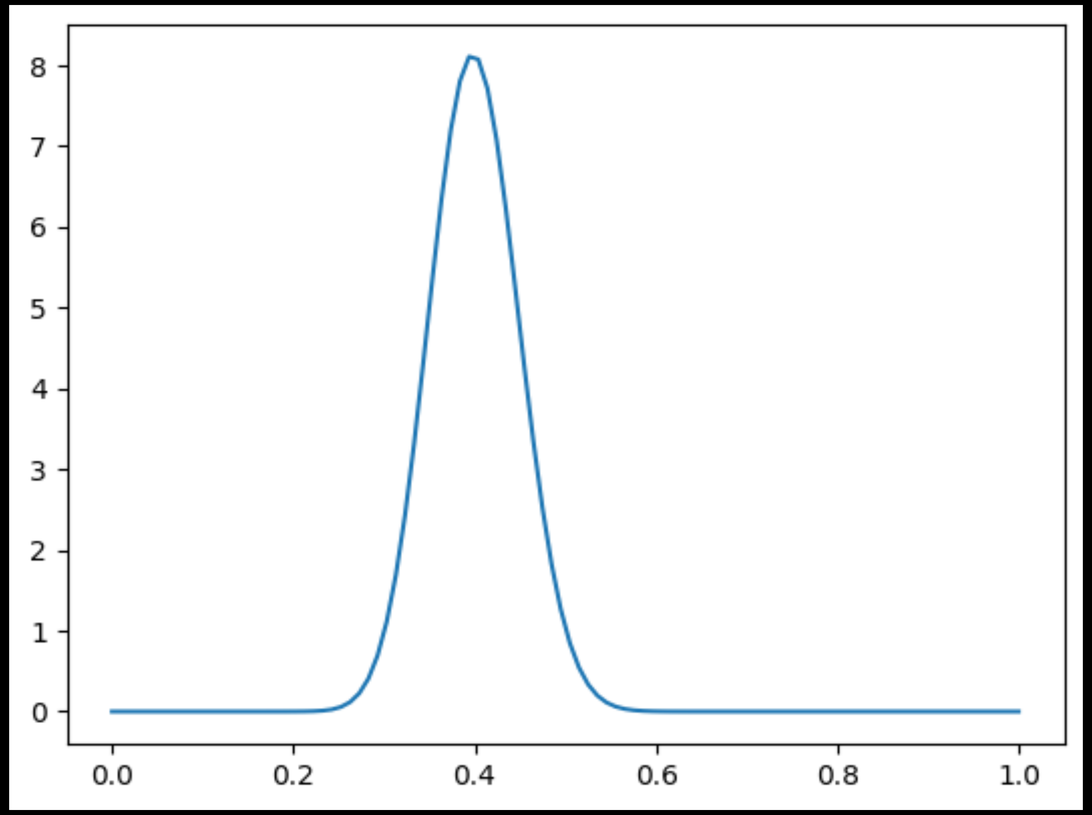
Beta(1,1)

Beta distribution



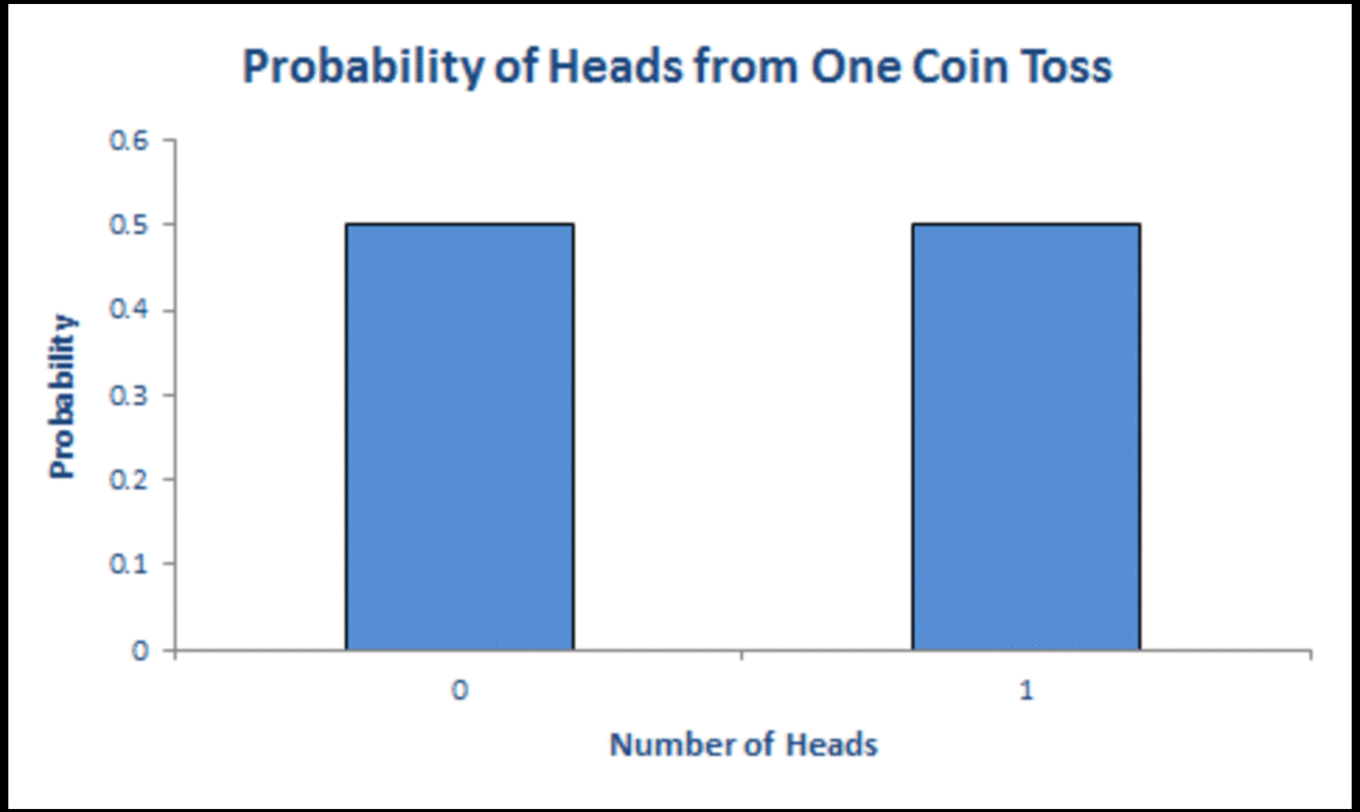
Beta(4,6)

Beta distribution



Beta(40,60)

Bernoulli distribution



Bernoulli distribution

$$f(k;p) = \begin{cases} p & \text{if } k = 1, \\ 1-p & \text{if } k = 0. \end{cases}$$

Beta-Bernoulli Bandits

Algorithm 5 Thompson Sampling Algorithm for Beta-Bernoulli Bandits

For each arm $i \in \{1, \ldots, K\}$, initialize $\alpha_i = \beta_i = 1$. ▷ Set $\text{Beta}(\alpha_i, \beta_i)$ prior for each p_i.
for $t = 1, 2, \ldots, T$ **do**:
    For each arm i, get sample $s_{i,t} \sim \text{Beta}(\alpha_i, \beta_i)$. ▷ Each is a float in $[0, 1]$.
    Pull arm $a_t = \arg\max_{i \in \{1,2,\ldots,K\}} s_{i,t}$. ▷ This "bakes in" exploration!
    Receive reward $r_t \sim \text{Ber}(p_{a_t})$.
    if $r_t == 1$ **then** $\alpha_{a_t} \leftarrow \alpha_{a_t} + 1$. ▷ Increment number of "successes".
    else if $r_t == 0$ **then** $\beta_{a_t} \leftarrow \beta_{a_t} + 1$. ▷ Increment number of "failures".

```python
@dataclass
class BetaPrior:
    a: int
    b: int
    p: float
```



```python
@dataclass
class BetaPrior:
    a: int
    b: int
    p: float
```



```python
@dataclass
class BetaPrior:
    a: int
    b: int
    p: float
```



We must sample something for this

```python
@dataclass
class BetaPrior:
    a: int
    b: int
    p: float
```

```python
priors = [BetaPrior(3, 7, 0.30), BetaPrior(3, 7, 0.35)]
```



Two priors

```python
@dataclass
class BetaPrior:
    a: int
    b: int
    p: float
```

```python
priors = [BetaPrior(3, 7, 0.30), BetaPrior(3, 7, 0.35)]
```

```python
arms = [stats.beta(p.a, p.b) for p in priors]
```



Two arms to pull

```python
@dataclass
class BetaPrior:
    a: int
    b: int
    p: float
```

```python
priors = [BetaPrior(3, 7, 0.30), BetaPrior(3, 7, 0.35)]
```

```python
arms = [stats.beta(p.a, p.b) for p in priors]
```

```python
sample = [bandit.rvs() for bandit in arms]
```



We get a chance

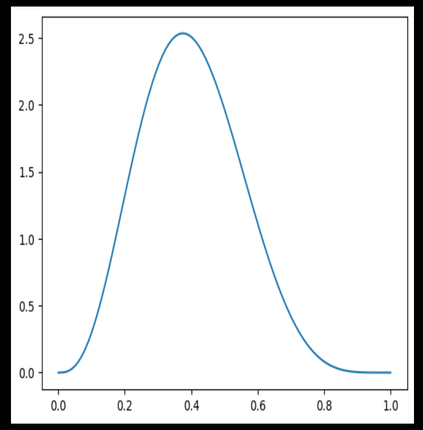
```python
@dataclass
class BetaPrior:
    a: int
    b: int
    p: float
```

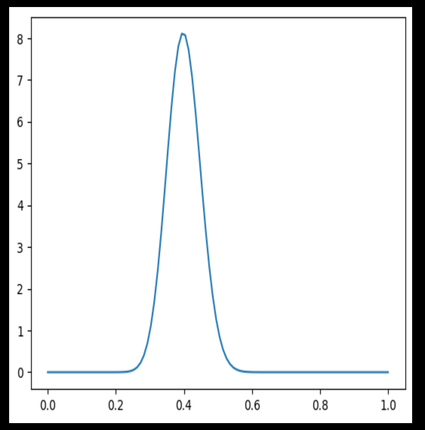
```python
priors = [BetaPrior(3, 7, 0.30), BetaPrior(3, 7, 0.35)]
```

```python
arms = [stats.beta(p.a, p.b) for p in priors]
```

```python
sample = [bandit.rvs() for bandit in arms]
```

If we are unsure, we explore





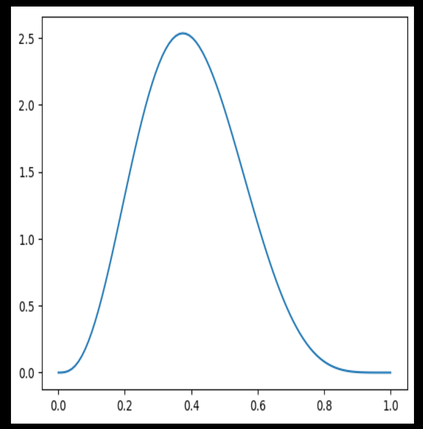
```python
@dataclass
class BetaPrior:
    a: int
    b: int
    p: float
```

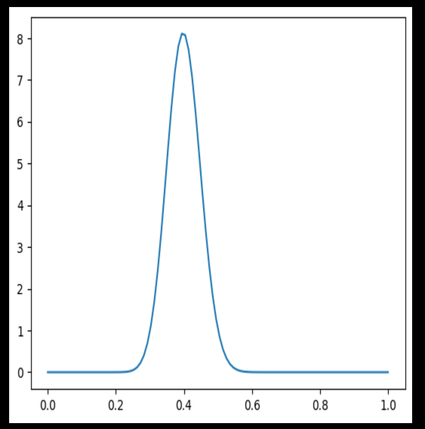
```python
priors = [BetaPrior(3, 7, 0.30), BetaPrior(3, 7, 0.35)]
```

```python
arms = [stats.beta(p.a, p.b) for p in priors]
```

```python
sample = [bandit.rvs() for bandit in arms]
```

If we are certain, we exploit





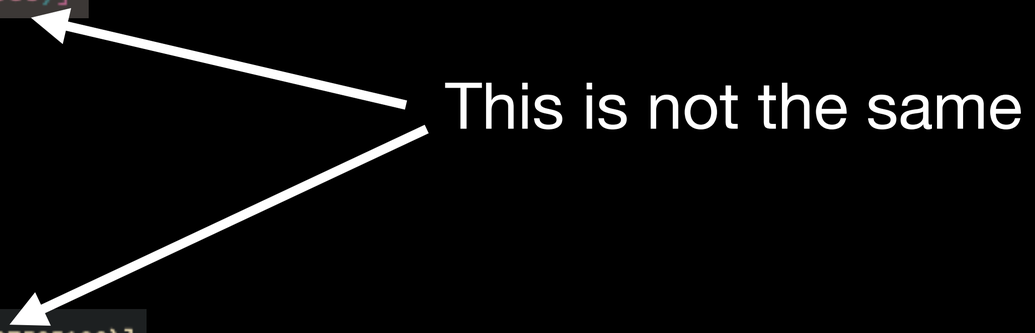
```python
@dataclass
class BetaPrior:
    a: int
    b: int
    p: float
```

```python
priors = [BetaPrior(3, 7, 0.30), BetaPrior(3, 7, 0.35)]
```

```python
arms = [stats.beta(p.a, p.b) for p in priors]
```

```python
sample = [bandit.rvs() for bandit in arms]
```

```
[np.float64(0.23518015098945705), np.float64(0.4471865397565186)]
```



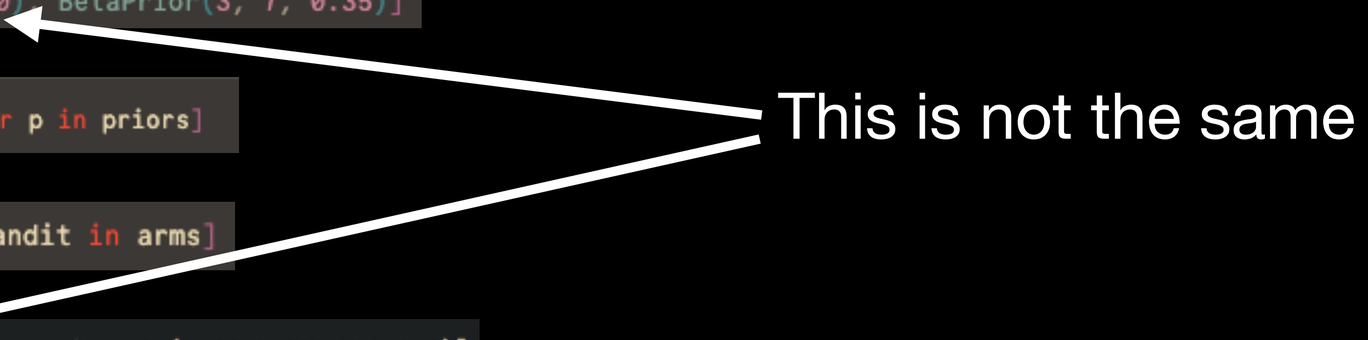
```python
@dataclass
class BetaPrior:
    a: int
    b: int
    p: float
```

```python
priors = [BetaPrior(3, 7, 0.30), BetaPrior(3, 7, 0.35)]
```

```python
arms = [stats.beta(p.a, p.b) for p in priors]
```

```python
sample = [bandit.rvs() for bandit in arms]
```

```
[np.float64(0.23518015098945705), np.float64(0.4471865397565186)]
```



Beta-Bernoulli Bandits

Algorithm 5 Thompson Sampling Algorithm for Beta-Bernoulli Bandits

For each arm $i \in \{1, \ldots, K\}$, initialize $\alpha_i = \beta_i = 1$. ▷ Set $\text{Beta}(\alpha_i, \beta_i)$ prior for each p_i.
for $t = 1, 2, \ldots, T$ **do**:
    For each arm i, get sample $s_{i,t} \sim \text{Beta}(\alpha_i, \beta_i)$. ▷ Each is a float in $[0, 1]$.
    Pull arm $a_t = \arg\max_{i \in \{1,2,\ldots,K\}} s_{i,t}$. ▷ This "bakes in" exploration!
    Receive reward $r_t \sim \text{Ber}(p_{a_t})$.
    if $r_t == 1$ **then** $\alpha_{a_t} \leftarrow \alpha_{a_t} + 1$. ▷ Increment number of "successes".
    else if $r_t == 0$ **then** $\beta_{a_t} \leftarrow \beta_{a_t} + 1$. ▷ Increment number of "failures".

```python
class Reward:
    def __init__(self, priors: list[BetaPrior]) -> None:
        self.p = [beta.p for beta in priors]
        self.bernoulli = [stats.bernoulli(beta.p) for beta in priors]

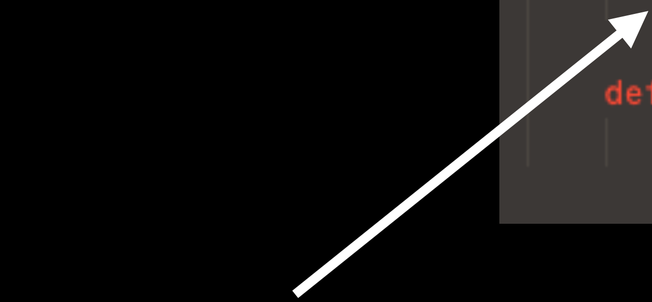
    def pay(self, arm: int) -> bool:
        return bool(self.bernoulli[arm].rvs())

    def __repr__(self) -> str:
        return f"Reward({self.p})"
```

```python
class Reward:
    def __init__(self, priors: list[BetaPrior]) -> None:
        self.p = [beta.p for beta in priors]
        self.bernoulli = [stats.bernoulli(beta.p) for beta in priors]

    def pay(self, arm: int) -> bool:
        return bool(self.bernoulli[arm].rvs())

    def __repr__(self) -> str:
        return f"Reward({self.p})"
```



You might win, or not

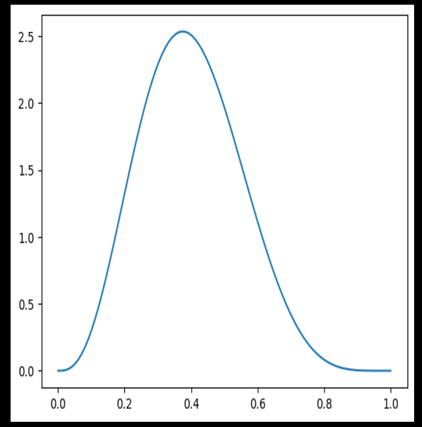
Update your belief

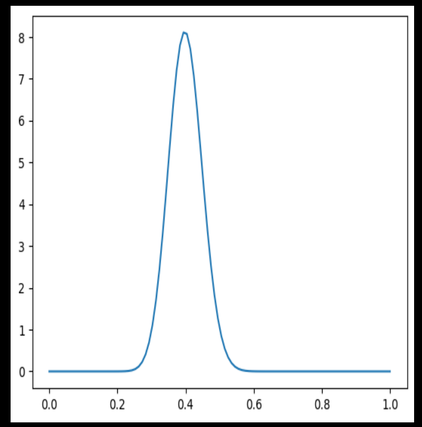
```python
class MultiArmedBandit:
    def __init__(self, priors: list[BetaPrior]):
        self.arms = [stats.beta(p.a, p.b) for p in priors]
        self.rewards = Reward(priors)

    def __repr__(self) -> str:
        args = [arm.args for arm in self.arms]
        return f"Bandit({args})"

    def pull(self) -> None:
        sample = [arm.rvs() for arm in self.arms]
        # the "luck" part of sampling helps us still explore other arms a bit
        arm = np.argmax(sample)
        reward = self.rewards.pay(arm)

        a, b = self.arms[arm].args
        if reward:
            # winning
            self.arms[arm].args = (a + 1, b)
        else:
            # loosing
            self.arms[arm].args = (a, b + 1)
```





```python
class MultiArmedBandit:
    def __init__(self, priors: list[BetaPrior]):
        self.arms = [stats.beta(p.a, p.b) for p in priors]
        self.rewards = Reward(priors)

    def __repr__(self) -> str:
        args = [arm.args for arm in self.arms]
        return f"Bandit({args})"

    def pull(self) -> None:
        sample = [arm.rvs() for arm in self.arms]
        # the "luck" part of sampling helps us still explore other arms a bit
        arm = np.argmax(sample)
        reward = self.rewards.pay(arm)

        a, b = self.arms[arm].args
        if reward:
            # winning
            self.arms[arm].args = (a + 1, b)
        else:
            # loosing
            self.arms[arm].args = (a, b + 1)
```

